

Web One — Architecture Overview

Containerised · Modular Monolith · Docker Compose

High-Level Architecture

The project is **containerised and service-oriented**, but not fully microservices in the strict sense. It is best described as a **modular monolith backend** paired with supporting infrastructure services, all orchestrated by Docker Compose on a single bridge network (*weebone_net*).

Services

Service	Image / Stack	Port(s)	Role
frontend	React 19 + Vite 6	5173	SPA served to browser
backend	Node.js + Express 5	5000	REST API, auth, business logic
ahmes_viewer	Separate web app	3000	Specialised file viewer
db	PostgreSQL 15	5432 (int)	Relational data store
minio	MinIO (S3-compat.)	9000 / 9001	Object / file storage
prisma_migrate	Node.js one-shot	—	Schema migration + seed

Is It Microservices?

Multiple deployable containers exist (frontend, backend, viewer, db, storage), which is **microservice-like operationally**. However, business logic is concentrated in one API service (*backend_webbone*), organised by routes/controllers — making the application layer a **modular monolith**, not split across independent domain services.

Stack Breakdown

Frontend

React 19 · Vite 6 · React Router 7 · Axios with JWT injection · Protected routes by role

```
const api = axios.create({ baseURL: import.meta.env.VITE_API_URL });
```

Backend

Node.js + Express 5 · Prisma ORM + PostgreSQL · JWT auth with role middleware · File handling via multer · LAN-aware CORS

Data Layer

PostgreSQL for relational domain data. Prisma models: **User · Client · Product · File · Group · GroupUser**

Storage Layer

Pluggable storage abstraction supporting **local**, **SMB**, and **MinIO / S3-compatible** backends. The current deployment uses MinIO.

Backend Module Structure

Route prefix	Module	Auth
/api/auth	login / register / profile	public
/api/products	product CRUD	JWT
/api/files	browse / upload / resumable	JWT
/admin	clients / users / groups / settings / metrics	JWT + role

```
app.use('/admin',      adminRoutes);    // JWT + role
app.use('/api/auth',   authRoutes);
app.use('/api/products', productRoutes); // JWT
app.use('/api/files',  filesRoutes);    // JWT
```

Runtime Request Flow

1. Browser hits **frontend** (:5173)
2. Frontend calls **backend** via VITE_API_URL + /api/...
3. Backend validates JWT and applies role / client scoping
4. Backend reads/writes metadata in **PostgreSQL** via Prisma
5. Backend stores/retrieves files through the **storage adapter** (MinIO / SMB / local)
6. Optional navigation to **Ahmes Viewer** (:3000) for specialised file viewing

Design Strengths

Easy deployment	Docker Compose on a single host — one command up/down.
Clear separation	Infra concerns (DB / storage / viewer) decoupled from business API.
Pluggable storage	Swap local / SMB / MinIO without touching controllers.
Straightforward RBAC	superadmin · admin · cliente roles enforced per route.